

学号		成绩	
----	--	----	--



沈阳建筑大学

编译原理上机报告

名 称 编译原理实验三
 学 院 信息与控制工程学院
 专 业 计算机科学与技术
 班 级 计算机 1902 班
 姓 名 蔡艺洵

2021年 11月 30日

一、上机目的

通过设计、编制、调试一个典型的语法分析程序，实现对词法分析程序所提供的单词序列进行语法检查和结构分析，进一步掌握常用的语法分析方法。

1、选择最有代表性的语法分析方法，如 LL(1) 语法分析程序、算符优先分析程序和 LR 分析程序，并至少完成两个题目。

2、选择对各种常见程序语言都用的语法结构，如赋值语句（尤指表达式）作为分析对象，并且与所选语法分析方法要比较贴切。

(1) 实验前的准备

按实验的目的和要求，编写语法分析程序，同时考虑相应的数据结构。

(2) 调试

调试例子应包括符合语法规则的算术表达式，以及分析程序能够判别的若干错例。

(3) 输出

对于所输入的算术表达式，不论对错，都应有明确的信息告诉外界。

(4) 扩充

有余力的同学，可适当扩大分析对象。譬如：

① 算术表达式中变量名可以是一般标识符，还可含一般常数、数组元素、函数调用等等。

② 除算术表达式外，还可扩充分析布尔、字符、位等不同类型的各种表达式。③加强语法检查，尽量多和确切地指出各种错误。

(5) 编写上机实习报告。

二、基本原理和上机步骤

在实验二中，我们进行的是自上而下分析法（递归下降分析程序），而在实验三中，我们要进行的是另外一个分析法—自下而上分析法。编译中存在着多种自下而上的分析法，但不管哪种自下而上的分析法都是按照移进—归约法的原理建立起来的一种语法分析方法。这边介绍三种方法。

确定的自上而下语法分析方法—预测分析法（LL(1)方法）：

预测分析法仅对 LL(1)文法适用。该方法的基本思想是

：从 S 出发，从左到右扫描源程序，每次通过向前查看 1 个字符，选择合适的产生式，生成句子的最左推导。

预测分析法使用下推自动机的方式实现。实现 LL(1)方法的结构如下：

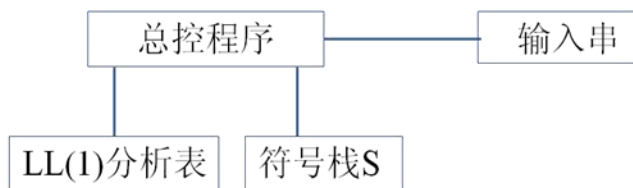


图 1 预测分析器模型图

预测分析器通过一个二维分析表和栈联合进行控制实现语法分析。分析器的总控程序大同小异，而 LL(1) 分析表却千差万别。所以，LL(1) 分析表是进行预测分析的核心。

要求一个文法的 LL(1) 分析表，必须判定该文法是否是 LL(1) 文法，若是，可通过求解文法的 $FIRST(\alpha)$ 和 $FOLLOW(U)$ 来构造 LL(1) 分析表；否则，必须先通过消除左递归和消除回溯来改造文法（不一定成功），使之成为 LL(1) 文法。

这边不再详细介绍如何构造 $FIRST(\alpha)$ 、 $FOLLOW(U)$ 、**SELECT 集**，但是这些知识都是我们必须储备的，也是本次上机实验需要去了解的知识。在完成上述的准备工作后，我们需要去构建 LL(1) 分析表，但是由于本次实验我选择的不是这个分析法，所以不再具体详细介绍，仅仅作为了解，不实现相关的程序。

自下而上分析技术—算符优先分析法：

这个也是作为了解，不在本次实验报告中实现。

自下而上分析技术—LR(K)方法

LR(K)方法是一种自下而上的语法分析方法，是当前最广义的无回溯的“移进-归约”方法。它根据栈中的符号串和向前查看的 $k(k \geq 0)$ 个输入符号，就能唯一确定分析器的动作是移进还是归约，以及用哪个产生式进行归约。

优点：文法适用范围广；识别效率高；查错能力强；可自动构造。

逻辑组成：总控程序+LR 分析表

这个是本次实验我选择采用的分析法，用代码去实现相关的功能。所以对相关的知识，做一个详细的介绍。

LR 分析器的结构：

一个 LR 分析器实际是一个带先进后出存储器（栈）的确定下推自动机，它由一个输入串、一个下推栈和一个带有分析表的总控程序组成。栈中存放着由“历史”和“展望”材料抽象而来的各种“状态”。任何时候，栈顶的状态都代表了整个的历史和已推测出的展望。为了有助于明确归约手续，我们把已归约出的文法符号串也同时放进栈里。LR 分析器的每一动作都由栈顶状态和当前输入符号所唯一确定。

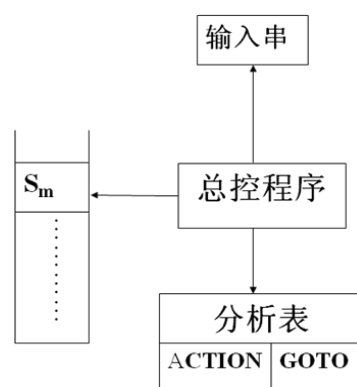


图 2 LR 分析器模型图

分析器的任何一次移动都是根据栈顶状态 S_m 和当前输入符号 a_i ，去查看 ACTION 表并执行 ACTION (S_m, a_i) 规定的动作，直至分析成功或失败。

LR 分析表有两个部分：动作部分 ACTION 和状态转换部分 GOTO。

ACTION[S,a]表明当前状态 S 面临输入符号 a 时应该采取的动作：

- 1、移入：将 S,y 的下一个状态 S 以及当前符号入栈。
- 2、归约：对栈顶的符号串按照某个规则进行归约。
- 3、接受：宣布输入符号串为一个句子。
- 4、报错：宣布输入符号串不是句子。

GOTO[S,U]表示当前状态 S 和非终结符号匹配的时候所转换到的下一个状态。

LR 总控程序：

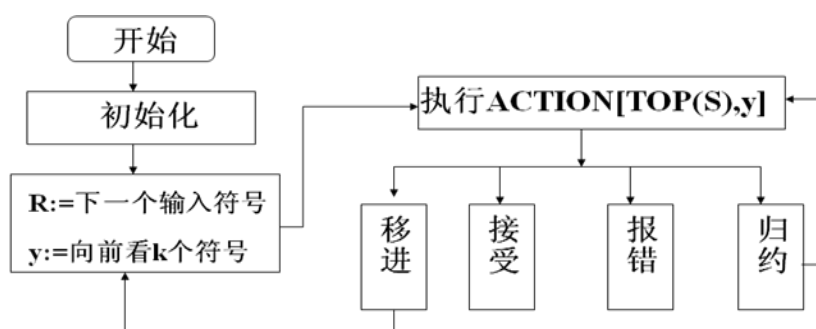


图3 LR 总控程序示意图

LR 分析器的工作过程是由总控程序根据分析表，使得分析器构型从一种构型向另一种构型变化的过程。初始构型：($S_0, a_1a_2...a_n \$$)， S_0 为分析器的初态， $\$$ 为输入串的括号。

分析过程的每步结果可表示为：($S_0X_1S_1...X_mS_m, a_ia_{i+1}...a_n\$$)。

分析器的下一次动作是由栈顶状态 S_m 和当前输入符号 a_i 所唯一确定的，即：执行 ACTION[S_m, a_i]规定的动作。经执行各种可能的动作后，分析器的构型可如下变化：

- 1、若 ACTION(S_m, a_i)=“移进 S”，则分析器构型变为 ($S_0X_1S_1...X_mS_ma_iS, a_{i+1}...a_n\$$)。
- 2、若 ACTION(S_m, a_i)=“归约 $A \rightarrow \beta$ ”，则分析器构型变为 ($S_0X_1S_1...X_{m-r}S_{m-r}AS, a_ia_{i+1}...a_n\$$)，其中 $S = GOTO(S_{m-r}, A)$ ， $|\beta|=r$ 。

3、若 ACTION(S_m, a_i)=“接受”，则分析成功，正常停止。

4、若 ACTION(S_m, a_i)=“ERROR”，语法出错，进行出错处理。

除了这些工作之外，开始编写代码之前，我还需要进行 LR 分析表的构造。

三、上机结果

程序输入/输出用例：

对下列文法，用 LR（1）分析法对任意输入的符号串进行分析：

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow i$

输出的格式如下：

(1)输入一以#结束的符号串(包括+—*/（）i#)：在此位置输入符号串

(2)输出过程如下：

表 1 输入示例表

步骤	状态栈	符号栈	剩余输入串	动作
1	0	#	i+i*i#	移进

(3)输入符号串为非法符号串(或者为合法符号串)

本次实验，采用 txt 文本来进行输入输出。lex_r.txt 表示输入文件，里面存储要识别的字符串，

如下图所示



图 4 输入文件

par_r.txt 是存储识别过程的文件，如下图所示：



图 5 输出文件

四、讨论与分析

对应本次上机实验编写的代码程序，先给出我实现代码的流程框图，如下所示：

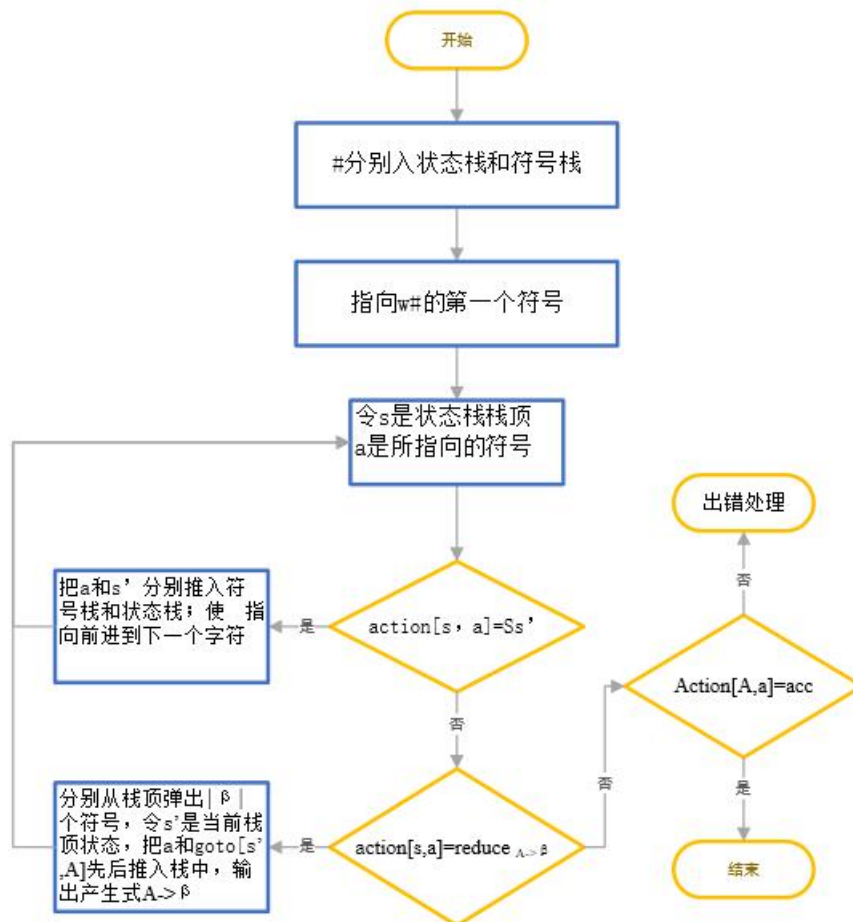


图 6 代码逻辑图

本次上机实验调试的程序运行得到的结果与预期的一致，说明了 LR 分析器的工作过程可看成是一个逐步识别所给文法规范句型活前缀的过程，在分析的每一步分析栈中的全部文法符号是当前规范句型的活前缀，且与当前栈顶状态相关联，因而可以利用有穷自动机去识别所给文法的所有规范句型的活前缀。本次上机实践，让我更加深刻的去了解 LR（0）分析法，这是我从三个分析法中选择自己要实现的分析法，之所以选择这个分析法去实现，是因为 LR（0）分析法的限制条件最少、适用范围最广。构造 LR 分析表的基本思想是：从给定的上下文无关文法直接构造识别文法所有规范句型活前缀的 DFA，然后再将 DFA 转换成一张 LR 分析表。

五、源程序清单

```
#include <fstream>
#include <iostream>
#include <stdlib.h>
#include <string>
struct code_val {
    char code;
    char val[20];
};
const char *p[] = { //产生式
    "S→E", "E→E+T", "E→T", "T→T*F", "T→F", "F→(E)", "F→i"
};
const char TNT[] = "+*() i#ETF"; //LR 分析表的字符
const int M[][9] = { // LR 分析表数字化, 列字符 + *() i#ETF 用数字 012345678 标识。
    { 0, 0, 4, 0, 5, 0, 1, 2, 3}, //0 表示出错, s4 用 4 表示。
    { 6, 0, 0, 0, 0, 99}, //Acc 用 99 表示
    {-2, 7, 0, -2, 0, -2}, //r2 用-2 表示
    {-4, -4, 0, -4, 0, -4},
    { 0, 0, 4, 0, 5, 0, 8, 2, 3},
    {-6, -6, 0, -6, 0, -6},
    { 0, 0, 4, 0, 5, 0, 0, 9, 3},
    { 0, 0, 4, 0, 5, 0, 0, 0, 10},
    { 6, 0, 0, 11},
    {-1, 7, 0, -1, 0, -1},
    {-3, -3, 0, -3, 0, -3},
    {-5, -5, 0, -5, 0, -5}
};
int col(char); //列定位函数原型
using namespace std;
void main()
{
    int state[50] = { 0 }; //状态栈初值
    char symbol[50] = { '#' }; //符号栈初值
    int top = 0; //栈顶指针初值
    ofstream cout("par_r.txt"); //语法分析结果输出至文件 par_r.txt
    ifstream cin("lex_r.txt"); // 从 lex_r.txt 中输入词法分析结果
    struct code_val t; //结构变量, 存放单词二元式。
    //t.val = "i+i*i#";
    cin >> t.code >> t.val; //读一单词
    int action;
    int i, j = 0; //输出时使用的计数器, 并非必要。
    cout << "step" << '\t' << "状态栈" << '\t' << "符号栈" << '\t' << "输入符号" << endl; //输出标题并非必要。
    do {
```

```

cout << j++ << ' ' << '\t'; //输出 step, 并非必要。
for (i = 0; i <= top; i++) cout << state[i]; cout << '\t'; //输出状态栈内容, 并非必要。
for (i = 0; i <= top; i++) cout << symbol[i]; //输出符号栈内容, 并非必要。
cout << '\t' << t.code << endl; //输出当前输入符号 (单词种别), 并非必要。
action = M[state[top]][col(t.code)];
if (action > 0 && action != 99) { //移进
    state[++top] = action;
    symbol[top] = t.code;
    t.code = t.val[0];
    for (int x = 0; x < 20; x++) {
        t.val[x] = t.val[x + 1];
    }
    //cin >> t.code >> t.val; //读一单词
}
else if (action < 0) { //归约
    if (strcmp(p[-action] + 3, "ε")) //ε 产生式的右部符号串长度为 0, 无需退栈。
        top = top - (strlen(p[-action]) - 3); // "→" 为汉字, 占二字节, 故减 3。
    state[top + 1] = M[state[top]][col(p[-action][0])]; //产生式左部符号
    symbol[++top] = p[-action][0];
}
else if (action == 99) { //接受
    cout << '\t' << "Acc" << endl;
    break;
}
else { //出错
    cout << "Err in main()>" << action << endl;
    break;
}
} while (1);
}

int col(char c) //将字符* ()i#ETf 分别转换为数字 012345678
{
    for (int i = 0; i < (int)strlen(TNT); i++)
        if (c == TNT[i]) return i;
    cout << "Err in col char>" << c << endl;
    exit(0); //终止程序运行
}

```